

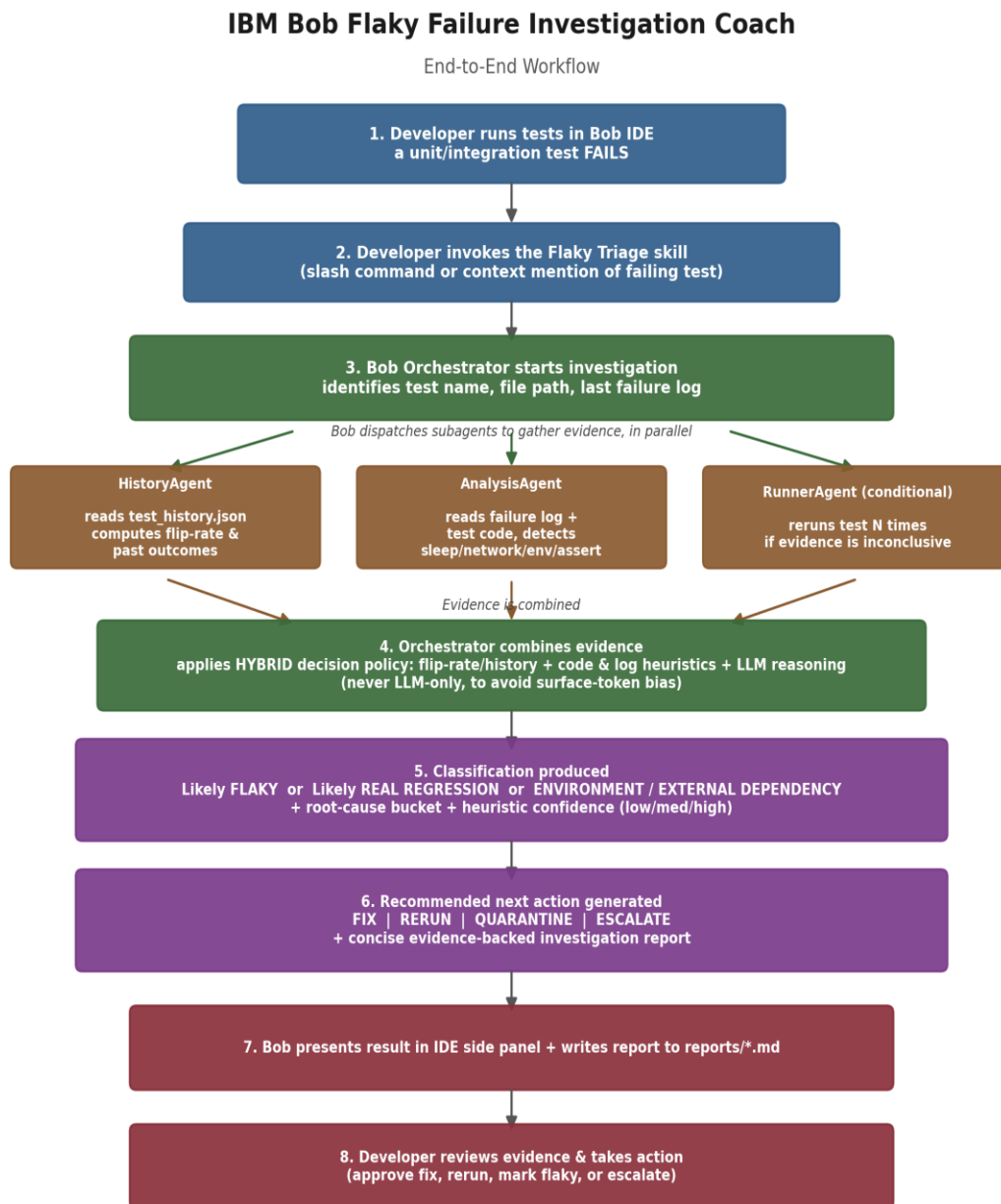
IBM Bob Flaky Failure Investigation Coach

Team Workflow Reference

1. Purpose

This document defines the exact end-to-end workflow our Bob agent follows when a unit or integration test fails. It is the shared reference for building, testing, and demoing the project — everyone should be building against this same flow.

2. Workflow Diagram



3. Step-by-Step Breakdown

- Step 1 — Developer runs the test suite inside Bob IDE. A specific unit/integration test fails.
- Step 2 — Developer invokes the Flaky Triage skill (via a slash command or by mentioning the failing test file/name).
- Step 3 — Bob's orchestrator starts the investigation: identifies the test name, file path, and the most recent failure log.
- Step 4 — Bob dispatches subagents to gather evidence in parallel (see table below): History, Analysis, and — only if needed — Runner.
- Step 5 — The orchestrator combines all evidence and applies a hybrid decision policy: historical flip-rate + code/log heuristics + LLM reasoning. The LLM is never used alone, to avoid the known weakness where AI models over-rely on surface patterns like 'sleep' or 'await' instead of real context.
- Step 6 — Bob produces a classification (flaky / real regression / environment / external dependency), a root-cause bucket, and a heuristic confidence level (low/medium/high — explicitly not a scientific score).
- Step 7 — Bob recommends one next action: FIX, RERUN, QUARANTINE, or ESCALATE, and writes a short evidence-backed report.
- Step 8 — Bob presents the result in the IDE side panel and saves the report to reports/<test_name>_triage.md.
- Step 9 — The developer reviews the evidence and takes the recommended action (approve a fix, rerun, mark the test flaky, or escalate).

4. Subagent Design

Three subagents divide the investigation work. This is what makes the project a genuine agentic workflow rather than a single chat prompt — and it is what the judging criteria specifically rewards.

Subagent	Input	Action	Output
HistoryAgent	Test name	Reads test_history.json; computes flip-rate and recent pass/fail pattern	Flip-rate, recent outcomes, error types
AnalysisAgent	Failure log + test/code file	Scans for signals: time.sleep, network calls, shared state, env vars, assertion type	Structured feature list (uses_sleep, error_type, etc.)
RunnerAgent (conditional)	Test name + N	Reruns the failing test N times only if History + Analysis are inconclusive	Pass/fail pattern across reruns

5. Decision Logic — Example Outcomes

The same workflow produces different outcomes depending on the evidence gathered. These are the five cases we should be able to demo clearly:

Case	Evidence pattern	Recommended action
Genuinely flaky	Mixed pass/fail history; uses sleep/timing/network; reruns show inconsistent result on same code	QUARANTINE (mark flaky) + fix later
Real regression	Stable pass history until a recent code change; consistent AssertionError; reruns all fail	FIX (Bob proposes a patch)
Environment failure	Fails on missing env var / config; consistent KeyError-type failure	ESCALATE to infra/CI owner or fix config

Case	Evidence pattern	Recommended action
External dependency	Intermittent ConnectionError tied to an external call	FIX test (mock the dependency); optionally QUARANTINE
Insufficient evidence	Very little history; inconsistent, unrecognized errors	ESCALATE with the gathered evidence log

6. What This Demonstrates in the Hackathon Demo

- A real multi-step agentic workflow (Agent mode + subagents), not a single-prompt wrapper.
- A clear before/after: manual log-reading and guesswork vs. a structured, evidence-backed recommendation.
- A hybrid, research-informed decision policy — not a naive 'ask the LLM' approach.
- A visible, demoable output: the IDE panel and the saved report.